

PYTHON FOR QUANT TRADERS · PART VI

Async & Live Trading — เทรดจริง แบบอัตโนมัติ

บทที่ 31–35: Async Fundamentals · asyncio Patterns · WebSocket &
Live Data · Order Execution · Live System Integration

จบ Part นี้ → ระบบ live trading อัตโนมัติที่รับ feed จริง ส่งออเดอร์จริง
พร้อม kill switch และ monitoring

ทำไม **PART** นี้ถึงสำคัญที่สุด?

Backtest บอกว่า strategy ทำกำไรได้ — แต่นั่นคือบน ข้อมูลอดีต ที่นิ่งอยู่ใน DataFrame

ในโลกจริง ราคาเปลี่ยนทุกมิลลิวินาที มีหลาย feed พร้อมกัน ต้องส่ง order ตรงเวลา ต้องจัดการ error ที่ไม่คาดคิด

- **Async** ให้ Python จัดการงานหลายอย่างพร้อมกันโดยไม่ต้องใช้ thread จำนวนมาก
- **WebSocket** รับ market data แบบ real-time แทน polling ทุก 1 วินาที
- **Order Execution** ส่ง order อย่างปลอดภัย มี rate limit และ retry
- **Kill Switch** หยุดระบบทันทีเมื่อเกิดเหตุฉุกเฉิน

▣

▣ **STOP — อ่านก่อนเริ่ม Part VI: Production Safety Checklist**

Part VI สอน async และ live trading กับเงินจริง ก่อนเดินหน้าให้ตอบ YES ทุกข้อ:

▣

1. **API Keys** อยู่ใน **.env file** — ห้าม hardcode ใน code เด็ดขาด
2. **ทดสอบบน Testnet ครบ 48+ ชั่วโมง** — Bybit Testnet / Binance Testnet
3. **Kill switch** พร้อม — มีปุ่มหรือ Telegram command หยุดระบบทันที
4. **Drawdown limit** ตั้งไว้แล้ว — เช่น หยุดอัตโนมัติเมื่อขาดทุน 5%
5. **Position size** ไม่เกิน 5% ของทุน ต่อ 1 trade สำหรับ system ใหม่
6. **Log ทุก order** — timestamp, price, size, status เก็บไว้อย่างน้อย 30 วัน
7. **Monitor** ตลอด 24 ชั่วโมงแรก — อย่าปล่อยให้ระบบทำงานโดยไม่มีคนดู

ถ้ายังไม่ครบ — ทำ Part VI ต่อเพื่อเรียนรู้ได้ แต่อย่าเชื่อมกับ real account จนกว่าจะผ่านทุกข้อ

Running Example ตลอด **Part VI — BTC Pair Strategy** แบบ **Live**

เราจะนำ PairStrategy จาก Part II/III มาเปลี่ยนให้รันแบบ real-time:

บท 31 → เข้าใจ async · บท 32 → asyncio patterns · บท 33 → WebSocket

feed

บท 34 → ส่ง order ผ่าน REST API · บท 35 → ระบบครบวงจรพร้อม deploy

คำเตือนสำคัญก่อนเริ่ม **Part VI**

- ทดสอบบน **Testnet** เสมอ — ยำนำ code จาก tutorial ไปรันบน real account โดยตรง
- ห้าม **hardcode API Key** — ใช้ environment variable หรือ secrets manager เท่านั้น
- ต้องมี **Kill Switch** — ทุกระบบ live ต้องหยุดได้ทันทีเมื่อกด Ctrl+C หรือรับ signal
- **Log ทุก order** — ถ้าไม่มี log จะตามสอบไม่ได้ว่าเกิดอะไรขึ้น
- **Monitor drawdown** — ตั้ง max drawdown threshold ให้ระบบหยุดอัตโนมัติ

บทที่ 31: Async Fundamentals

แก่นของบทนี้

Async คือวิธีให้ Python "รอ" อะไรบางอย่าง (เช่น network) โดยไม่บล็อก thread — ทำให้ทำงานหลายอย่างพร้อมกันใน thread เดียว เหมือนพนักงานร้านอาหาร 1 คนที่รับออเดอร์หลายโต๊ะพร้อมกัน แทนที่จะยืนรอโต๊ะแรกก่อนแล้วค่อยไปโต๊ะถัดไป

31.1 ปัญหาของ Blocking Code

ใน live trading เราต้องรับราคาจากหลาย exchange พร้อมกัน หาก code ทำงานแบบ blocking จะช้ามาก

```
# ===== # แบบ BLOCKING - รอทีละตัว (ช้า) #
===== import time
import requests def fetch_price_blocking(exchange: str) -> float: # จำลองการเรียก API (ใช้เวลา 1 วินาที) time.sleep(1) prices = {"binance": 65000.0, "bybit": 65010.0, "okx": 64990.0, "bitget": 65005.0} return prices[exchange] def get_all_prices_blocking(): start = time.time() results = {} for ex in ["binance", "bybit", "okx", "bitget"]: results[ex] = fetch_price_blocking(ex) # รอทีละตัว! elapsed = time.time() - start print(f"Blocking: ใช้เวลา {elapsed:.1f}s") return results
get_all_prices_blocking()
```

► OUTPUT

Blocking: ใช้เวลา 4.0s

```
# ===== # แบบ ASYNC - รอพร้อมกัน (เร็ว) #
===== import asyncio
import time async def fetch_price_async(exchange: str) -> float: # asyncio.sleep ไม่บล็อก event loop await asyncio.sleep(1) prices = {"binance": 65000.0, "bybit": 65010.0, "okx": 64990.0, "bitget": 65005.0} return prices[exchange] async def get_all_prices_async(): start = time.time() # gather รัน coroutine ทั้งหมดพร้อมกัน results = await asyncio.gather( fetch_price_async("binance"), fetch_price_async("bybit"), fetch_price_async("okx"), fetch_price_async("bitget"), ) elapsed = time.time() - start print(f"Async: ใช้เวลา {elapsed:.1f}s") exchanges =
```

```
["binance", "bybit", "okx", "bitget"] return dict(zip(exchanges, results))
# รัน event loop prices = asyncio.run(get_all_prices_async()) print(prices)
```

► OUTPUT

Async: ใช้เวลา 1.0s

```
{'binance': 65000.0, 'bybit': 65010.0, 'okx': 64990.0, 'bitget': 65005.0}
```

สรุปความแตกต่าง: Blocking vs Async

ด้าน	Blocking	Async
เวลา 4 API calls (1s each)	4 วินาที	~1 วินาที
CPU ขณะรอ network	นอนรออยู่เฉยๆ	ทำงานอื่นระหว่างรอ
Concurrency model	ต้องใช้ thread/process	1 thread พอ
Memory overhead	สูง (thread ละ ~8MB)	ต่ำ (coroutine ละ ~KB)
เหมาะกับ	งาน CPU-heavy	งาน I/O-heavy (network, disk)

31.2 async def และ await — หัวใจของ Async

```
# ===== # กฎพื้นฐาน 3
ข้อ # ===== # 1.
function ที่มี "await" ข้างใน ต้องเป็น "async def" async def fetch_ticker(symbol:
str) -> dict: await asyncio.sleep(0.1) # await บอกว่า "รอตตรงนี้ได้ไปทำอื่นก่อน"
return {"symbol": symbol, "price": 65000.0, "volume": 1234.5} # 2. "await"
ใช้ได้แค่ภายใน "async def" เท่านั้น async def main(): ticker = await
fetch_ticker("BTCUSDT") # ถูกต้อง print(ticker) # 3. เรียก async function จาก
synchronous code ด้วย asyncio.run() asyncio.run(main()) #
===== # coroutine คือ
object ที่ยังไม่รัน - ต้อง await หรือ schedule #
===== coro =
fetch_ticker("BTCUSDT") # สร้าง coroutine object print(type(coro)) # <class
'coroutine'> - ยังไม่รัน! result = asyncio.run(coro) # รัน coroutine
print(result)
```

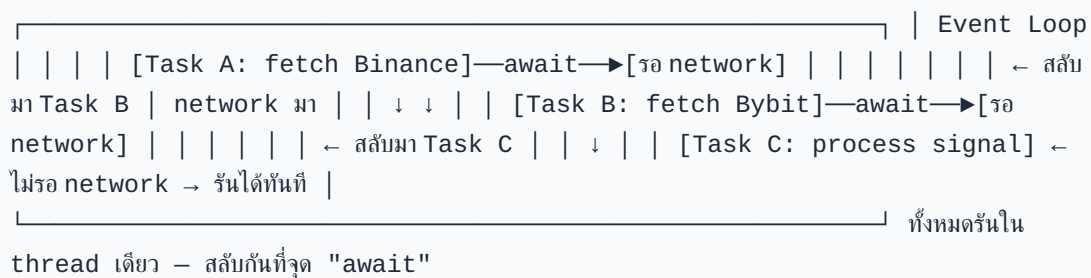
► OUTPUT

```
<class 'coroutine'>
{'symbol': 'BTCUSDT', 'price': 65000.0, 'volume': 1234.5}
```

31.3 Event Loop — หัวใจของ asyncio

Event loop คือ "ผู้จัดการ" ที่วิ่งอยู่เบื้องหลัง คอยตรวจว่า coroutine ไหนพร้อมรันแล้ว และสลับการทำงานระหว่าง coroutines

Event Loop Lifecycle:



```
import asyncio
async def demonstrate_event_loop():
    """แสดงการสลับ context ระหว่าง coroutines"""
    async def task_a():
        print("Task A: เริ่มต้น")
        await asyncio.sleep(0.2) # ยกมือบอก event loop: "ไปทำอื่นก่อน"
        print("Task A: กลับมาแล้ว")
        return "A done"
    async def task_b():
        print("Task B: เริ่มต้น")
        await asyncio.sleep(0.1) # รอน้อยกว่า A → จบก่อน
        print("Task B: กลับมาแล้ว")
        return "B done"
    async def task_c():
        print("Task C: ไม่รอ network เลย")
        # await asyncio.sleep(0) บอก event loop ให้สลับสักครั้ง
        await asyncio.sleep(0)
        print("Task C: จบแล้ว")
        return "C done"
    results = await asyncio.gather(task_a(), task_b(), task_c())
    print(f"Results: {results}")
asyncio.run(demonstrate_event_loop())
```

► OUTPUT

```
Task A: เริ่มต้น
Task B: เริ่มต้น
Task C: ไม่รอ network เลย
Task C: จบแล้ว
Task B: กลับมาแล้ว
Task A: กลับมาแล้ว
Results: ['A done', 'B done', 'C done']
```

31.4 ทำไม Blocking Code ถึงอันตรายใน Async

อันตราย: ห้ามใช้ **Blocking call** ภายใน **async function**

```
# ผิด - time.sleep() บล็อก event loop ทั้งหมด! async def
bad_fetch(exchange: str) -> float: time.sleep(1) # บล็อก! coroutine อื่น
หยุดรอด้วย return 65000.0 # ผิด - requests.get() บล็อก event loop! async
def bad_rest_call(url: str): resp = requests.get(url) # บล็อก! ใช้
aiohttp แทน return resp.json() # ✓ ถูกต้อง - asyncio.sleep ไม่บล็อก async
def good_fetch(exchange: str) -> float: await asyncio.sleep(1) #
yield control กลับ event loop return 65000.0 # ✓ ถูกต้อง - ใช้ aiohttp สำหรับ
HTTP import aiohttp async def good_rest_call(url: str): async with
aiohttp.ClientSession() as session: async with session.get(url) as
resp: return await resp.json()
```

Blocking (อย่าใช้ใน async)	Async alternative
<code>time.sleep(n)</code>	<code>await asyncio.sleep(n)</code>
<code>requests.get(url)</code>	<code>await session.get(url)</code> (aiohttp)
<code>open(file).read()</code>	<code>await aiofiles.open(file).read()</code>
<code>socket.recv()</code>	WebSocket library async
CPU-heavy loop	<code>await loop.run_in_executor(None, fn)</code>

ตัวอย่าง: Async price aggregator สำหรับ BTC pair

```
import asyncio from dataclasses import dataclass from datetime
import datetime @dataclass class Ticker: exchange: str symbol: str
price: float timestamp: datetime async def
fetch_ticker_mock(exchange: str, symbol: str, base_price: float,
delay: float) -> Ticker: """จำลอง REST API call""" await
asyncio.sleep(delay) import random price = base_price +
random.uniform(-50, 50) return Ticker(exchange, symbol, price,
datetime.utcnow()) async def aggregate_btc_prices() -> dict[str,
Ticker]: """ดึงราคาBTC จากหลาย exchange พร้อมกัน""" tickers = await
asyncio.gather( fetch_ticker_mock("binance", "BTCUSDT", 65000.0,
0.08), fetch_ticker_mock("bybit", "BTCUSDT", 65005.0, 0.12),
fetch_ticker_mock("okx", "BTCUSDT", 64995.0, 0.10), ) result =
{t.exchange: t for t in tickers} prices = [t.price for t in tickers]
spread = max(prices) - min(prices) mid = sum(prices) / len(prices)
print(f"Mid: ${mid:,.1f} Spread: ${spread:.1f}") return result
asyncio.run(aggregate_btc_prices())
```

► OUTPUT

Mid: \$65,002.3 Spread: \$47.2

► แบบฝึกหัด: เพิ่ม **timeout** ให้ **fetch** — ถ้า **exchange** ไม่ตอบใน 0.5s ให้ข้ามไป

```
async def fetch_with_timeout(exchange: str, symbol: str, base_price:
float, delay: float, timeout: float = 0.5) -> Ticker | None: """ดึงราคา
พร้อม timeout""" try: return await
asyncio.wait_for( fetch_ticker_mock(exchange, symbol, base_price,
delay), timeout=timeout ) except asyncio.TimeoutError: print(f"[WARN]
{exchange} timeout หลัง {timeout}s - ข้าม") return None async def
aggregate_with_timeout(): tasks = [ fetch_with_timeout("binance",
"BTCUSDT", 65000.0, 0.08), fetch_with_timeout("bybit", "BTCUSDT",
65005.0, 0.60), # จะ timeout fetch_with_timeout("okx", "BTCUSDT",
64995.0, 0.10), ] results = await asyncio.gather(*tasks) valid =
{r.exchange: r for r in results if r is not None} print(f"ได้ข้อมูล
{len(valid)}/3 exchange") return valid
asyncio.run(aggregate_with_timeout())
```


บทที่ 32: asyncio Patterns

แก่นของบทนี้

asyncio มี building blocks สำคัญ 4 อย่างสำหรับระบบ trading จริง:

gather() สำหรับ concurrent fetch, **Queue** สำหรับ decouple producer/consumer, **wait_for()** สำหรับ timeout, และ **Task** สำหรับ background jobs ที่ cancel ได้

32.1 asyncio.gather() — รันพร้อมกัน

```
import asyncio
async def fetch_ohlcv(exchange: str, symbol: str, timeframe: str = "1m") -> list:
    """จำลองดึง OHLCV candle ล่าสุด"""
    await asyncio.sleep(0.1)
    import random
    p = 65000 + random.uniform(-200, 200)
    return [exchange, symbol, timeframe, {"open": p-10, "high": p+20, "low": p-30, "close": p, "volume": random.uniform(100, 500)}]

async def fetch_all_ohlcv():
    """ดึง OHLCV จากหลาย exchange/timeframe พร้อมกัน"""
    pairs = [ ("binance", "BTCUSDT", "1m"), ("bybit", "BTCUSDT", "1m"), ("binance", "BTCUSDT", "5m"), # timeframe อื่นด้วย ("bybit", "BTCUSDT", "5m"), ]
    # gather ทุก coroutine พร้อมกัน
    results = await asyncio.gather( *[fetch_ohlcv(ex, sym, tf) for ex, sym, tf in pairs], return_exceptions=True # แทนที่จะ raise พันที่ให้เกิด exception ใน result )
    for r in results:
        if isinstance(r, Exception):
            print(f"[ERROR] {r}")
        else:
            ex, sym, tf, candle = r
            print(f"{ex:8s} {sym} {tf:3s}: close={candle['close']:, .1f}")
    asyncio.run(fetch_all_ohlcv())
```

► OUTPUT

```
binance  BTCUSDT 1m : close=65,187.3
bybit    BTCUSDT 1m : close=64,823.6
binance  BTCUSDT 5m : close=65,041.2
bybit    BTCUSDT 5m : close=65,112.8
```

32.2 asyncio.Queue — Decouple Producer และ Consumer

ใน live trading มี 2 ส่วนที่ทำงานเร็วต่างกัน: **feed** รับข้อมูลเร็ว และ **signal engine** คำนวณช้ากว่า Queue เป็นตัวกลางที่ buffer ข้อมูลไว้

```
Producer (WebSocket feed)  Consumer (Signal Engine)
_____ tick มาทุก 10ms
```

—put→ `asyncio.Queue` —get→ ค่าผ่าน signal ไม่ต้องรอ consumer (buffer ≤ 1000 items) ส่ง order

```
import asyncio import random from dataclasses import dataclass, field from
datetime import datetime @dataclass class PriceTick: exchange: str symbol:
str price: float timestamp: datetime =
field(default_factory=datetime.utcnow) async def price_producer(queue:
asyncio.Queue, exchange: str, symbol: str, n_ticks: int = 20): """จำลอง
WebSocket feed ส่ง tick เข้า queue""" price = 65000.0 for i in range(n_ticks):
price += random.uniform(-50, 50) tick = PriceTick(exchange, symbol,
round(price, 2)) await queue.put(tick) print(f"[PROD] {exchange:8s} put
tick #{i+1:2d}: {price:,.1f}") await asyncio.sleep(0.05) # จำลอง tick
interval # ส่ง sentinel เพื่อบอก consumer ว่าจบแล้ว await queue.put(None) async def
signal_consumer(queue: asyncio.Queue, window: int = 5): """คำนวณ signal จาก
tick ที่รับจาก queue""" prices = [] processed = 0 while True: tick = await
queue.get() # รอจนกว่าจะมี item ใน queue if tick is None: # sentinel – จบ
queue.task_done() break prices.append(tick.price) queue.task_done()
processed += 1 if len(prices) >= window: ma = sum(prices[-window:]) /
window latest = prices[-1] signal = "LONG" if latest > ma else ("SHORT" if
latest < ma else "HOLD") print(f"[CONS] tick #{processed:2d}:
price={latest:,.1f} " f"MA{window}={ma:,.1f} → {signal}") async def
producer_consumer_demo(): queue = asyncio.Queue(maxsize=100) # buffer สูงสุด
100 items # รัน producer และ consumer พร้อมกัน await
asyncio.gather( price_producer(queue, "binance", "BTCUSDT", n_ticks=10),
signal_consumer(queue, window=5), ) print("จบ demo")
asyncio.run(producer_consumer_demo())
```

32.3 `asyncio.wait_for()` — Timeout

```
import asyncio async def place_order_mock(order_id: str, delay: float =
0.3) -> dict: """จำลอง REST API order placement""" await
asyncio.sleep(delay) return {"order_id": order_id, "status": "FILLED",
"avg_price": 65000.0} async def place_order_with_timeout(order_id: str,
timeout: float = 1.0) -> dict | None: """ส่ง order พร้อม timeout – ถ้าช้าเกินไปถือว่า
fail""" try: result = await asyncio.wait_for( place_order_mock(order_id,
delay=0.5), timeout=timeout ) print(f"[OK] Order {order_id}:
{result['status']}") return result except asyncio.TimeoutError:
print(f"[WARN] Order {order_id}: timeout หลัง {timeout}s") # ต้อง query status
ภายหลัง – order อาจ filled หรือ rejected return None
asyncio.run(place_order_with_timeout("ORD-001", timeout=1.0))
```

► OUTPUT

[OK] Order ORD-001: FILLED

32.4 asyncio.Task — Background Jobs และ Cancellation

```
import asyncio
async def heartbeat(interval: float = 1.0): """ส่ง heartbeat ทุก interval วินาที – background task"""
    count = 0
    while True:
        count += 1
        print(f"[HB] Heartbeat #{count}")
        await asyncio.sleep(interval)
    async def main_trading_loop(duration: float = 3.5): """Main loop – รัน heartbeat เป็น background task"""
        # สร้าง Task – รันใน background ทันที
        hb_task = asyncio.create_task(heartbeat(1.0), name="heartbeat")
        print(f"[MAIN] เริ่ม trading loop ({duration}s)")
        await asyncio.sleep(duration)
        # จำลองการเทรด # Cancel background task เมื่อจบ
        hb_task.cancel()
        try:
            await hb_task
        except asyncio.CancelledError:
            print("[MAIN] Heartbeat task ถูก cancel แล้ว")
        print("[MAIN] จบ")
    asyncio.run(main_trading_loop())
```

► OUTPUT

```
[MAIN] เริ่ม trading loop (3.5s)
[HB] Heartbeat #1
[HB] Heartbeat #2
[HB] Heartbeat #3
[MAIN] Heartbeat task ถูก cancel แล้ว
[MAIN] จบ
```

32.5 asyncio.Semaphore — จำกัดจำนวน Concurrent Requests

Exchange มักจำกัด rate ไว้ เช่น "ส่ง request ได้ไม่เกิน 10 req/s" — Semaphore ช่วยควบคุม

```
import asyncio
async def fetch_historical(symbol: str, date: str, sem: asyncio.Semaphore) -> dict: """ดึงข้อมูล historical พร้อม rate limiting"""
    async with sem: # เข้า critical section – ไม่เกิน N พร้อมกัน
        await asyncio.sleep(0.1) # จำลอง API call
        return {"symbol": symbol, "date": date, "close": 65000.0}
    async def bulk_fetch_historical(symbols: list[str], dates: list[str]) -> list[dict]: """ดึงข้อมูลหลายวันพร้อมกัน แต่จำกัดไม่เกิน 5 request พร้อมกัน"""
        sem = asyncio.Semaphore(5) # อนุญาต 5 concurrent requests
        tasks = [fetch_historical(sym, date, sem) for sym in symbols for date in dates]
        print(f"ดึงข้อมูล {len(tasks)} records ด้วย semaphore=5")
        results = await asyncio.gather(*tasks)
        return list(results)
    asyncio.run(bulk_fetch_historical(symbols=["BTCUSDT", "ETHUSDT"],
        dates=["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04", "2024-01-05"] ))
```

► แบบฝึกหัด: สร้าง **MultiQueue** สำหรับรับข้อมูลจากหลาย **exchange** แล้วรวมเข้า **Queue** เดียว

```
async def multi_exchange_fan_in(exchanges: list[str], symbol: str,
n_ticks: int = 5) -> None: """รับ tick จากหลาย exchange → รวมเข้า 1 output
queue""" output_q: asyncio.Queue[PriceTick] = asyncio.Queue() async
def single_feed(exchange: str): base = 65000.0 + {"binance": 0,
"bybit": 10, "okx": -5}[exchange] for _ in range(n_ticks): price =
base + random.uniform(-30, 30) await output_q.put(PriceTick(exchange,
symbol, round(price, 2))) await asyncio.sleep(random.uniform(0.05,
0.15)) await output_q.put(None) # sentinel ต่อ exchange async def
consumer(n_producers: int): finished = 0 while finished < n_producers:
tick = await output_q.get() if tick is None: finished += 1 else:
print(f"[FAN-IN] {tick.exchange:8s}: {tick.price:,.1f}")
output_q.task_done() await asyncio.gather( *[single_feed(ex) for ex in
exchanges], consumer(len(exchanges)) )
asyncio.run(multi_exchange_fan_in( ["binance", "bybit", "okx"],
"BTCUSDT" ))
```

บทที่ 33: WebSocket & Live Data

แก่นของบทนี้

WebSocket คือ connection แบบ "เปิดค้างไว้" ระหว่าง client และ server — ต่างจาก HTTP ที่ต้อง request ทุกครั้ง Exchange ส่ง tick/candle ให้เราทันทีที่มีการเปลี่ยนแปลง ทำให้ latency ต่ำกว่า polling มาก แต่ต้องจัดการ disconnect, reconnect, และ parse message เองทั้งหมด

33.1 Binance Futures WebSocket — รูปแบบ Message จริง

Binance Futures ส่ง kline (candle) stream ในรูปแบบ JSON นี้ เมื่อ subscribe ที่ `wss://fstream.binance.com/ws/btcusdt@kline_1m`

```
# ตัวอย่าง kline message จาก Binance Futures WebSocket (format จริง)
BINANCE_KLINE_MSG = { "e": "kline", # event type "E": 1705123456789, #
  event time (ms) "s": "BTCUSDT", # symbol "k": { "t": 1705123440000, # kline
    start time (ms) "T": 1705123499999, # kline close time (ms) "s": "BTCUSDT",
    # symbol "i": "1m", # interval "f": 100, # first trade ID "L": 200, # last
    trade ID "o": "65000.10", # open "c": "65123.45", # close (current price)
    "h": "65200.00", # high "l": "64950.00", # low "v": "123.456", # base asset
    volume (BTC) "n": 1250, # number of trades "x": False, # is this kline
    closed? False = ยังไม่ปิด candle "q": "8023456.78", # quote asset volume (USDT)
    "V": "61.234", # taker buy base volume "Q": "3987654.32", # taker buy quote
    volume } } # ตัวอย่าง bookTicker message จาก Binance (best bid/ask)
BINANCE_BOOKTICKER_MSG = { "e": "bookTicker", "u": 400900217, # order book
  updateId "s": "BTCUSDT", "b": "65000.00", # best bid price "B": "5.000", #
  best bid qty "a": "65001.00", # best ask price "A": "3.500", # best ask qty
  "T": 1705123456789, # transaction time "E": 1705123456790, # event time }
```

33.2 Bybit WebSocket — รูปแบบ Message

```
# Bybit V5 WebSocket - wss://stream.bybit.com/v5/public/linear # Subscribe
message: BYBIT_SUBSCRIBE = { "op": "subscribe", "args": ["kline.1.BTCUSDT"]
# kline.{interval}.{symbol} } # Kline message จาก Bybit V5: BYBIT_KLINE_MSG
= { "topic": "kline.1.BTCUSDT", "data": [ { "start": 1705123440000, # start
  time ms "end": 1705123499999, # end time ms "interval": "1", # interval in
  minutes "open": "65000.10", "close": "65123.45", "high": "65200.00", "low":
  "64950.00", "volume": "123.456", # base volume (BTC) "turnover":
```

```
"8023456.78", # quote volume (USDT) "confirm": False, # False = candle ยังไม่  
ปิด "timestamp": 1705123456789 } ], "ts": 1705123456789, "type": "snapshot"  
# หรือ "delta" }
```

33.3 WebSocket Client ที่ Reconnect อัตโนมัติ

```
import asyncio import json import logging import random import websockets  
from dataclasses import dataclass from datetime import datetime from typing  
import Callable, Optional logger = logging.getLogger(__name__) @dataclass  
class Candle: exchange: str symbol: str interval: str open: float high:  
float low: float close: float volume: float is_closed: bool timestamp:  
datetime class BinanceWebSocketClient: """ Binance Futures WebSocket client  
- reconnect อัตโนมัติด้วย exponential backoff - parse kline messages - ส่ง Candle  
object เข้า callback """ WS_URL = "wss://fstream.binance.com/ws"  
MAX_RECONNECT_DELAY = 60.0 # วินาที INITIAL_RECONNECT_DELAY = 1.0 def  
__init__(self, symbol: str, interval: str, on_candle: Callable[[Candle],  
None], queue: Optional[asyncio.Queue] = None): self.symbol = symbol.lower()  
self.interval = interval self.on_candle = on_candle self.queue = queue  
self._stop_event = asyncio.Event() self._reconnect_delay =  
self.INITIAL_RECONNECT_DELAY @property def stream_url(self) -> str: return  
f"{self.WS_URL}/{self.symbol}@kline_{self.interval}" def _parse_kline(self,  
raw: dict) -> Optional[Candle]: """แปลง Binance kline message เป็น Candle  
object""" try: k = raw["k"] return Candle( exchange = "binance", symbol =  
raw["s"], interval = k["i"], open = float(k["o"]), high = float(k["h"]),  
low = float(k["l"]), close = float(k["c"]), volume = float(k["v"]),  
is_closed = bool(k["x"]), timestamp = datetime.utcfromtimestamp(raw["E"] /  
1000) ) except (KeyError, ValueError, TypeError) as e:  
logger.error(f"Binance parse error: {e} | raw={str(raw)[:200]}") return  
None async def _connect_and_listen(self): """เชื่อมต่อและฟัง messages - 1  
connection attempt""" logger.info(f"Connecting to {self.stream_url}") async  
with websockets.connect( self.stream_url, ping_interval=20, # ส่ง ping ทุก 20s  
ping_timeout=10, close_timeout=5, ) as ws: logger.info(f"Connected:  
{self.symbol}@kline_{self.interval}") self._reconnect_delay =  
self.INITIAL_RECONNECT_DELAY # reset async for raw_msg in ws: if  
self._stop_event.is_set(): break try: msg = json.loads(raw_msg) if  
msg.get("e") == "kline": candle = self._parse_kline(msg) if candle is not  
None: # ถ้า parse ไม่ได้ self.on_candle(candle) # ถ้ามี queue ให้ put ด้วย if  
self.queue is not None: await self.queue.put(candle) except  
json.JSONDecodeError as e: logger.error(f"JSON decode error: {e}") except  
KeyError as e: logger.error(f"Unexpected message format: {e} |  
msg={raw_msg[:200]}") async def run(self): """รัน WebSocket พร้อม exponential  
backoff reconnect""" while not self._stop_event.is_set(): try: await  
self._connect_and_listen() except websockets.exceptions.ConnectionClosed as  
e: logger.warning(f"Connection closed: {e.code} {e.reason}") except  
websockets.exceptions.WebSocketException as e: logger.error(f"WebSocket  
error: {e}") except OSError as e: logger.error(f"Network error: {e}") if  
self._stop_event.is_set(): break # Exponential backoff + jitter ป้องกัน  
thundering herd logger.info(f"Reconnecting in {self._reconnect_delay:.1f}  
s...") await asyncio.sleep(self._reconnect_delay) self._reconnect_delay =  
min( self._reconnect_delay * 2, self.MAX_RECONNECT_DELAY ) +  
random.uniform(0, 1.0) logger.info("WebSocket client stopped") def
```

```

stop(self): """หยุด client อย่างสะอาด""" self._stop_event.set() class
BybitWebSocketClient: """ Bybit V5 WebSocket client endpoint: wss://
stream.bybit.com/v5/public/linear """ WS_URL = "wss://stream.bybit.com/v5/
public/linear" MAX_RECONNECT_DELAY = 60.0 INITIAL_RECONNECT_DELAY = 1.0 def
__init__(self, symbol: str, interval: str, on_candle: Callable[[Candle],
None], queue: Optional[asyncio.Queue] = None): self.symbol = symbol.upper()
self.interval = interval self.on_candle = on_candle self.queue = queue
self._stop_event = asyncio.Event() self._reconnect_delay =
self.INITIAL_RECONNECT_DELAY def _parse_kline(self, msg: dict) ->
list[Candle]: """แปลง Bybit kline message – อาจมีหลาย candle ใน 1 message"""
candles = [] for k in msg.get("data", []): try:
candles.append(Candle( exchange = "bybit", symbol = self.symbol, interval =
self.interval, open = float(k["open"]), high = float(k["high"]), low =
float(k["low"]), close = float(k["close"]), volume = float(k["volume"]),
is_closed = bool(k.get("confirm", False)), timestamp =
datetime.utcfromtimestamp(k["start"] / 1000) )) except (KeyError,
ValueError, TypeError) as e: logger.error(f"Bybit parse error: {e} |
k={str(k)[:200]}") return candles async def _connect_and_listen(self):
logger.info(f"Connecting to Bybit: {self.symbol} kline.{self.interval}")
async with websockets.connect( self.WS_URL, ping_interval=20,
ping_timeout=10, ) as ws: # ส่ง subscribe message sub_msg =
json.dumps({ "op": "subscribe", "args": [f"kline.{self.interval}.
{self.symbol}"] }) await ws.send(sub_msg) self._reconnect_delay =
self.INITIAL_RECONNECT_DELAY async for raw_msg in ws: if
self._stop_event.is_set(): break try: msg = json.loads(raw_msg) if
msg.get("op") == "subscribe": logger.info(f"Bybit subscribe:
{msg.get('ret_msg')}") continue if "topic" in msg and "kline" in
msg["topic"]: for candle in self._parse_kline(msg): self.on_candle(candle)
if self.queue is not None: await self.queue.put(candle) except
(json.JSONDecodeError, KeyError) as e: logger.error(f"Parse error: {e}")
async def run(self): while not self._stop_event.is_set(): try: await
self._connect_and_listen() except
(websockets.exceptions.WebSocketException, OSError) as e:
logger.warning(f"Connection error: {e}") if not self._stop_event.is_set():
await asyncio.sleep(self._reconnect_delay) self._reconnect_delay =
min( self._reconnect_delay * 2, self.MAX_RECONNECT_DELAY ) +
random.uniform(0, 1.0) def stop(self): self._stop_event.set()

```

33.4 Dual-Feed WebSocket — รับ Binance + Bybit พร้อมกัน

Running Example: รับ kline จาก Binance และ Bybit พร้อมกัน → ส่งเข้า Queue เดียว

นี่คือ core ของ live pair strategy — เราต้องการราคาจากทั้ง 2 exchange ในเวลาเดียวกันเพื่อคำนวณ spread

```

import asyncio import json import logging from collections import deque
from typing import Optional logger = logging.getLogger(__name__) async def
run_dual_feed(symbol: str = "BTCUSDT", interval: str = "1", max_candles:
int = 100, run_seconds: float = 30.0): """ รัน Binance + Bybit WebSocket พร้อม
กัน ส่ง candle ทั้งสองเข้า shared queue หยุดเองหลัง run_seconds วินาที (สำหรับ demo) """
shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=500) stop_event
= asyncio.Event() # Buffer สำหรับคำนวณ spread candle_buf: dict[str, deque] =
{ "binance": deque(maxlen=max_candles), "bybit":
deque(maxlen=max_candles), } def on_candle(candle: Candle): # Callback รัน
synchronously – ใช้นัด log if candle.is_closed:
logger.debug(f"[{candle.exchange}] Closed candle " f"{candle.symbol}
close={candle.close}") # สร้าง clients binance_client =
BinanceWebSocketClient( symbol=symbol, interval=f"{interval}s",
on_candle=on_candle, queue=shared_queue ) bybit_client =
BybitWebSocketClient( symbol=symbol, interval=interval,
on_candle=on_candle, queue=shared_queue ) async def process_candles():
"""Consumer: คำนวณ spread เมื่อได้ candle ไปด้วยทั้งสอง""" while not
stop_event.is_set(): try: candle = await
asyncio.wait_for( shared_queue.get(), timeout=1.0 ) except
asyncio.TimeoutError: continue if candle.is_closed:
candle_buf[candle.exchange].append(candle.close) # คำนวณ spread เมื่อมีข้อมูลทั้งสอง
exchange if (len(candle_buf["binance"]) > 0 and len(candle_buf["bybit"]) >
0): b_price = candle_buf["binance"][-1] y_price = candle_buf["bybit"][-1]
spread = y_price - b_price logger.info(f"Spread: {spread:+.2f} "
f"(Binance={b_price:.1f}, Bybit={y_price:.1f})") shared_queue.task_done()
async def stopper(): await asyncio.sleep(run_seconds) stop_event.set()
binance_client.stop() bybit_client.stop() logger.info("Stop event set") #
รันทุก task พร้อมกัน await asyncio.gather( binance_client.run(),
bybit_client.run(), process_candles(), stopper(), return_exceptions=True )
# ใช้งาน: # asyncio.run(run_dual_feed("BTCUSDT", "1", run_seconds=300))

```

ข้อควรระวัง: WebSocket ใน Production

- **ตรวจสอบ timestamp** — ถ้า message เก่ากว่า 5 วินาที อาจเป็น stale data → อย่าเทรด
- **Bybit** ต้องการ **re-subscribe** หลัง reconnect — ต้องส่ง subscribe message ใหม่ทุกครั้ง
- **Binance** จำกัด **10 connections** ต่อ IP — ระวังถ้ารัน multiple instances
- **ใช้ testnet ก่อน**: Binance testnet: <wss://stream.binancefuture.com/ws> , Bybit testnet: <wss://stream-testnet.bybit.com/v5/public/linear>

► แบบฝึกหัด: เพิ่ม **staleness check** — ถ้า **candle** เก่ากว่า 30 วินาที ให้ **log warning**

```
from datetime import datetime, timezone def check_staleness(candle:
Candle, max_age_seconds: float = 30.0) -> bool: """คืน True ถ้า candle ยัง
สด (ไม่เกิน max_age_seconds)""" now =
datetime.now(timezone.utc).replace(tzinfo=None) age = (now -
candle.timestamp).total_seconds() if age > max_age_seconds:
logger.warning( f"[STALE] {candle.exchange} {candle.symbol} " f"candle
age={age:.1f}s > {max_age_seconds}s - ช้า" ) return False return True
# ใช้ใน process_candles(): # if candle.is_closed and
check_staleness(candle, max_age_seconds=30): #
candle_buf[candle.exchange].append(candle.close)
```

บทที่ 34: Order Execution

แก่นของบทนี้

การส่ง order จริงซับซ้อนกว่า backtest มาก: ต้องมี HMAC signature, จัดการ rate limit, retry เมื่อ network error (แต่ไม่ retry ถ้า order ถูก reject), และติดตาม state ของ order ตั้งแต่ PENDING จนถึง FILLED หรือ CANCELLED

34.1 Order State Machine

Order State Machine:

Diagram illustrating the Order State Machine transitions:

```
graph LR
    PENDING -- "(สร้าง order object ใน code)" --> OPEN
    OPEN -- "(exchange รับ order แล้ว)" --> PARTIAL
    PARTIAL -- "fill" --> FILLED
    PARTIAL -- "fully filled" --> FILLED
    FILLED -- "(ซื้อ/ขายครบแล้ว)" --> FILLED
    PENDING -- "Error path: PENDING" --> REJECTED
    OPEN -- "Error path: OPEN" --> ERROR
    REJECTED -- "(invalid params, insufficient balance)" --> REJECTED
    ERROR -- "(network lost, exchange down)" --> ERROR
```

```
from enum import Enum, auto from dataclasses import dataclass, field from
typing import Optional import time class OrderStatus(Enum): PENDING =
auto() # สร้างใน code แต่ยังไม่ส่ง OPEN = auto() # ส่งไป exchange แล้วรอ fill
PARTIALLY_FILLED = auto() FILLED = auto() # fill ครบแล้ว CANCELLED = auto()
REJECTED = auto() # exchange ปฏิเสธ ERROR = auto() # network/unknown error
class OrderSide(Enum): BUY = "Buy" SELL = "Sell" class OrderType(Enum):
MARKET = "MARKET" LIMIT = "LIMIT" @dataclass class Order: client_order_id:
str # ID ที่เราสร้างเอง (UUID หรือ timestamp) symbol: str side: OrderSide
order_type: OrderType qty: float price: Optional[float] = None # None สำหรับ
market order # Fields ที่ exchange จะส่งกลับมา exchange_order_id: Optional[str] =
None status: OrderStatus = OrderStatus.PENDING filled_qty: float = 0.0
avg_price: float = 0.0 created_at: float = field(default_factory=time.time)
updated_at: Optional[float] = None reject_reason: Optional[str] = None
@property def is_terminal(self) -> bool: """คืน True ถ้า order จบแล้ว (ไม่สามารถ
เปลี่ยนได้อีก)""" return self.status in ( OrderStatus.FILLED,
OrderStatus.CANCELLED, OrderStatus.REJECTED, OrderStatus.ERROR, ) @property
def remaining_qty(self) -> float: return self.qty - self.filled_qty def
__repr__(self): return (f"Order({self.client_order_id} "
```

```
f"{self.side.value} {self.qty} {self.symbol} " f"@ {'MKT' if self.price is
None else self.price} " f"[{self.status.name}]]")
```

34.2 REST API Wrapper พร้อม Rate Limiting และ Retry

```
import asyncio import hashlib import hmac import json import os import time
import logging from urllib.parse import urlencode from typing import
Optional import aiohttp logger = logging.getLogger(__name__) class
BybitOrderClient: """ Bybit V5 REST API wrapper - Rate limiting ด้วย
asyncio.Semaphore - HMAC-SHA256 signature - Retry with exponential backoff
สำหรับ network error - ไม่ retry ถ้า exchange reject (4xx) """ # Testnet:
https://api-testnet.bybit.com BASE_URL = "https://api.bybit.com"
RECV_WINDOW = 5000 # ms def __init__(self, api_key: Optional[str] = None,
api_secret: Optional[str] = None, testnet: bool = True, max_concurrent: int
= 5): # ✓ โหลดจาก env var - ห้าม hardcode! self.api_key = api_key or
os.environ.get("BYBIT_API_KEY", "") self.api_secret = api_secret or
os.environ.get("BYBIT_API_SECRET", "") if testnet: self.BASE_URL =
"https://api-testnet.bybit.com" logger.warning("ใช้ TESTNET - ไม่ใช่ real
money") self._sem = asyncio.Semaphore(max_concurrent) self._session:
Optional[aiohttp.ClientSession] = None async def __aenter__(self):
self._session = aiohttp.ClientSession() return self async def
__aexit__(self, *args): if self._session: await self._session.close() def
_sign(self, params: dict) -> str: """สร้าง HMAC-SHA256 signature""" timestamp
= str(int(time.time() * 1000)) params_str =
urlencode(sorted(params.items())) payload = f"{timestamp}{self.api_key}
{self.RECV_WINDOW}{params_str}" signature =
hmac.new( self.api_secret.encode("utf-8"), payload.encode("utf-8"),
hashlib.sha256 ).hexdigest() return signature, timestamp async def
_request(self, method: str, path: str, params: dict = None, max_retries:
int = 3) -> dict: """ส่ง signed request พร้อม retry""" params = params or {}
signature, timestamp = self._sign(params) headers = { "X-BAPI-API-KEY":
self.api_key, "X-BAPI-SIGN": signature, "X-BAPI-TIMESTAMP": timestamp, "X-
BAPI-RECV-WINDOW": str(self.RECV_WINDOW), "Content-Type": "application/
json", } url = f"{self.BASE_URL}{path}" for attempt in range(max_retries):
async with self._sem: # rate limiting try: if method == "GET": async with
self._session.get( url, params=params, headers=headers,
timeout=aiohttp.ClientTimeout(total=10) ) as resp: data = await resp.json()
else: async with self._session.post( url, json=params, headers=headers,
timeout=aiohttp.ClientTimeout(total=10) ) as resp: data = await resp.json()
# Bybit ret_code: 0 = OK, อื่นๆ = error if data.get("retCode") != 0: ret_msg
= data.get("retMsg", "unknown") ret_code = data.get("retCode") # Error ที่ไม่
ควร retry NO_RETRY_CODES = { 10001, # Parameter error 10004, # Sign check
failed 110007, # Insufficient balance 110013, # Position is in liquidation
110025, # Position not exist } if ret_code in NO_RETRY_CODES: raise
ValueError( f"Exchange rejected [{ret_code}]: {ret_msg}" )
logger.warning(f"API error [{ret_code}]: {ret_msg}, " f"attempt
{attempt+1}/{max_retries}") return data except (aiohttp.ClientError,
asyncio.TimeoutError) as e: if attempt == max_retries - 1: raise delay = 2
** attempt logger.warning(f"Network error: {e}, retry in {delay}s") await
asyncio.sleep(delay) raise RuntimeError(f"Failed after {max_retries}
attempts") async def place_market_order(self, symbol: str, side: OrderSide,
```

```

qty: float, client_order_id: str) -> Order: """"สั่ง market order"""" order =
Order( client_order_id=client_order_id, symbol=symbol, side=side,
order_type=OrderType.MARKET, qty=qty, ) try: resp = await
self._request("POST", "/v5/order/create", { "category": "linear", "symbol":
symbol, "side": side.value, "orderType": "Market", "qty": str(qty),
"orderLinkId": client_order_id, }) if resp.get("retCode") == 0: result =
resp["result"] order.exchange_order_id = result.get("orderId") order.status
= OrderStatus.OPEN logger.info(f"Order placed: {order}") else: order.status
= OrderStatus.REJECTED order.reject_reason = resp.get("retMsg")
logger.error(f"Order rejected: {order.reject_reason}") except ValueError as
e: order.status = OrderStatus.REJECTED order.reject_reason = str(e)
logger.error(f"Order rejected by exchange: {e}") except Exception as e:
order.status = OrderStatus.ERROR order.reject_reason = str(e)
logger.error(f"Order error: {e}") order.updated_at = time.time() return
order async def place_limit_order(self, symbol: str, side: OrderSide, qty:
float, price: float, client_order_id: str) -> Order: """"สั่ง limit order""""
order = Order( client_order_id=client_order_id, symbol=symbol, side=side,
order_type=OrderType.LIMIT, qty=qty, price=price, ) try: resp = await
self._request("POST", "/v5/order/create", { "category": "linear", "symbol":
symbol, "side": side.value, "orderType": "Limit", "qty": str(qty), "price":
str(price), "timeInForce": "PostOnly", # maker order เท่านั้น "orderLinkId":
client_order_id, }) if resp.get("retCode") == 0: result = resp["result"]
order.exchange_order_id = result.get("orderId") order.status =
OrderStatus.OPEN logger.info(f"Limit order placed: {order}") else:
order.status = OrderStatus.REJECTED order.reject_reason =
resp.get("retMsg") except ValueError as e: order.status =
OrderStatus.REJECTED order.reject_reason = str(e) except Exception as e:
order.status = OrderStatus.ERROR order.reject_reason = str(e)
order.updated_at = time.time() return order async def cancel_order(self,
symbol: str, client_order_id: str) -> bool: """"ยกเลิก order"""" resp = await
self._request("POST", "/v5/order/cancel", { "category": "linear", "symbol":
symbol, "orderLinkId": client_order_id, }) success = resp.get("retCode") ==
0 if success: logger.info(f"Cancelled order {client_order_id}") else:
logger.warning(f"Cancel failed: {resp.get('retMsg')}") return success async
def get_order_status(self, symbol: str, client_order_id: str) ->
Optional[Order]: """"ดึง order status จาก exchange"""" resp = await
self._request("GET", "/v5/order/realtime", { "category": "linear",
"symbol": symbol, "orderLinkId": client_order_id, }) if
resp.get("retCode") != 0: return None items = resp["result"].get("list",
[]) if not items: return None item = items[0] STATUS_MAP = { "New":
OrderStatus.OPEN, "PartiallyFilled": OrderStatus.PARTIALLY_FILLED,
"Filled": OrderStatus.FILLED, "Cancelled": OrderStatus.CANCELLED,
"Rejected": OrderStatus.REJECTED, } order = Order( client_order_id =
client_order_id, symbol = symbol, side = OrderSide.BUY if item["side"] ==
"Buy" else OrderSide.SELL, order_type = OrderType.MARKET if
item["orderType"] == "Market" else OrderType.LIMIT, qty =
float(item["qty"]), price = float(item["price"]) if item.get("price") else
None, exchange_order_id = item["orderId"], status =
STATUS_MAP.get(item["orderStatus"], OrderStatus.ERROR), filled_qty =
float(item.get("cumExecQty", 0)), avg_price = float(item.get("avgPrice",
0)), ) return order

```

34.3 ทดสอบบน Testnet

```
import asyncio import uuid async def demo_order_flow(): """ ทดสอบ order flow บน Bybit Testnet ต้องตั้ง env var: export BYBIT_API_KEY=your_testnet_key export BYBIT_API_SECRET=your_testnet_secret """ async with BybitOrderClient(testnet=True) as client: # สร้าง unique client order ID coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}" print(f"Placing market BUY 0.001 BTCUSDT (coid={coid})") order = await client.place_market_order( symbol = "BTCUSDT", side = OrderSide.BUY, qty = 0.001, client_order_id = coid, ) print(f"Result: {order}") # รอให้ order settle await asyncio.sleep(1.0) # ดึง status updated = await client.get_order_status("BTCUSDT", coid) if updated: print(f"Updated status: {updated.status.name}, " f"filled={updated.filled_qty}, avg={updated.avg_price}") # asyncio.run(demo_order_flow())
```

ข้อควรระวัง: Order Execution ในโลกจริง

- ห้าม **retry** ทุก **error** — ถ้า network timeout แล้ว retry → order อาจถูกส่ง 2 ครั้ง ใช้ client_order_id เพื่อ idempotency
- **Market order slip** ได้ — ราคา fill อาจต่างจากราคาตลาดมาก โดยเฉพาะ pair ที่ volume น้อย
- ตรวจสอบ **balance** ก่อนส่ง **order** — error 110007 (insufficient balance) ไม่ควรเกิดถ้าระบบออกแบบดี
- **Log** ทุก **order response** รวมถึง timestamp, exchange_order_id, และ status

► แบบฝึกหัด: เขียน **OrderTracker class** ที่เก็บ **history** ของทุก **order** และ **query** ได้

```
from collections import defaultdict from typing import Dict, List
class OrderTracker: """เก็บ history ของทุก order – query ได้ด้วย symbol หรือ
status""" def __init__(self): self._orders: Dict[str, Order] = {} #
coid -> Order self._by_symbol: Dict[str, List[str]] =
defaultdict(list) def track(self, order: Order) -> None:
self._orders[order.client_order_id] = order if order.client_order_id
not in self._by_symbol[order.symbol]:
self._by_symbol[order.symbol].append(order.client_order_id) def
update(self, order: Order) -> None: """อัปเดต order ที่มีอยู่แล้ว""" if
order.client_order_id in self._orders:
self._orders[order.client_order_id] = order def get(self, coid: str) -
> Optional[Order]: return self._orders.get(coid) def open_orders(self,
symbol: str = None) -> List[Order]: all_orders =
list(self._orders.values()) open_list = [o for o in all_orders if
o.status in (OrderStatus.PENDING, OrderStatus.OPEN,
OrderStatus.PARTIALLY_FILLED)] if symbol: open_list = [o for o in
open_list if o.symbol == symbol] return open_list def
filled_pnl_summary(self, symbol: str = None) -> dict: orders = [o for
o in self._orders.values() if o.status == OrderStatus.FILLED] if
symbol: orders = [o for o in orders if o.symbol == symbol] buys = [o
for o in orders if o.side == OrderSide.BUY] sells = [o for o in orders
if o.side == OrderSide.SELL] return { "total_filled": len(orders),
"buy_count": len(buys), "sell_count": len(sells), } def
__repr__(self): return (f"OrderTracker(total={len(self._orders)}, "
f"open={len(self.open_orders())})")
```

บทที่ 35: Live System Integration

แก่นของบทนี้

ประกอบทุกอย่างเข้าด้วยกันเป็น live trading system ที่ production-ready: logging ที่ structured, health check loop, kill switch ที่ทำงานได้จริง, graceful shutdown, และ pre-flight checklist ก่อน go-live

35.1 Structured Logging

ใน live system logging ต้องเป็น structured (JSON) เพื่อ search และ filter ง่าย — ไม่ใช่แค่ print string

```
import logging import json import sys from datetime import datetime,
timezone class StructuredFormatter(logging.Formatter): """ Formatter ที่
output JSON - ง่ายต่อการ search ด้วย jq หรือ ELK stack """ def format(self, record:
logging.LogRecord) -> str: log_obj = { "ts":
datetime.now(timezone.utc).isoformat(), "level": record.levelname,
"logger": record.name, "msg": record.getMessage(), } # เพิ่ม extra fields ถ้ามี
for key in ("symbol", "order_id", "price", "side", "pnl", "error"): if
hasattr(record, key): log_obj[key] = getattr(record, key) if
record.exc_info: log_obj["exception"] =
self.formatException(record.exc_info) return json.dumps(log_obj,
ensure_ascii=False) def setup_logging(level: str = "INFO", log_file: str =
None) -> logging.Logger: """ตั้งค่า logging สำหรับ live system""" logger =
logging.getLogger("live_trading") logger.setLevel(getattr(logging,
level.upper())) # Console handler console =
logging.StreamHandler(sys.stdout)
console.setFormatter(StructuredFormatter()) logger.addHandler(console) #
File handler (ถ้าระบุ) if log_file: fh = logging.FileHandler(log_file,
encoding="utf-8") fh.setFormatter(StructuredFormatter())
logger.addHandler(fh) return logger # ใช้งาน logger =
setup_logging(level="INFO", log_file="live_trading.log") # Log พร้อม extra
context logger.info("Order placed", extra={ "symbol": "BTCUSDT",
"order_id": "ORD-001", "side": "BUY", "price": 65000.0 })
```

► OUTPUT

```
{"ts": "2024-01-15T10:30:00.123Z", "level": "INFO", "logger": "live_trading", "msg": "Ord
```

35.2 Kill Switch ด้วย asyncio.Event

```
import asyncio import signal import logging logger =
logging.getLogger("live_trading") class KillSwitch: """ Kill switch สำหรับหยุด
ระบบ live trading รองรับ: Ctrl+C, SIGTERM (Docker stop), programmatic
(drawdown exceeded) """ def __init__(self): self._event = asyncio.Event()
self._reason: str = "" def trigger(self, reason: str = "manual"): """ทริกเกอร์
kill switch""" self._reason = reason self._event.set()
logger.critical(f"KILL SWITCH TRIGGERED: {reason}") async def wait(self):
"""รอจนกว่า kill switch จะทำงาน""" await self._event.wait() @property def
is_set(self) -> bool: return self._event.is_set() @property def
reason(self) -> str: return self._reason def setup_signal_handlers(self,
loop: asyncio.AbstractEventLoop): """ตั้งค่า OS signal handlers""" def
_handle_signal(sig_name: str): logger.warning(f"Received {sig_name}")
self.trigger(f"OS signal: {sig_name}") # add_signal_handler รองรับเฉพาะ Unix/
Linux/macOS (ไม่ทำงานบน Windows) if sys.platform != "win32": for sig in
(signal.SIGINT, signal.SIGTERM): loop.add_signal_handler( sig, lambda
s=sig: _handle_signal(s.name) ) else: # Windows: ใช้ signal.signal() แทน
(synchronous) import signal as _signal _signal.signal(_signal.SIGINT,
lambda s, f: _handle_signal("SIGINT"))
```

35.3 Health Check Loop

```
import asyncio import time from dataclasses import dataclass, field
@dataclass class SystemHealth: last_binance_tick: float = 0.0 # timestamp
ของ tick ล่าสุดจาก Binance last_bybit_tick: float = 0.0 last_order_time: float =
0.0 total_pnl: float = 0.0 drawdown_pct: float = 0.0 peak_equity: float =
0.0 current_equity: float = 0.0 order_errors: int = 0 feed_disconnects: int
= 0 def update_equity(self, equity: float): if equity > self.peak_equity:
self.peak_equity = equity self.current_equity = equity if self.peak_equity
> 0: self.drawdown_pct = (equity - self.peak_equity) / self.peak_equity
async def health_check_loop(health: SystemHealth, kill_switch: KillSwitch,
check_interval: float = 5.0, max_drawdown: float = -0.05, max_feed_silence:
float = 30.0): """ ตรวจสอบสุขภาพระบบทุก check_interval วินาที หยุดอัตโนมัติถ้า: - drawdown
เกิน max_drawdown - ไม่ได้รับ feed นานกว่า max_feed_silence วินาที """ logger =
logging.getLogger("live_trading.health") while not kill_switch.is_set:
await asyncio.sleep(check_interval) now = time.time() # ตรวจสอบ drawdown if
health.drawdown_pct < max_drawdown: kill_switch.trigger( f"Max drawdown
exceeded: {health.drawdown_pct:.2%} < {max_drawdown:.2%}" ) break # ตรวจสอบ
feed silence binance_silence = now - health.last_binance_tick bybit_silence
= now - health.last_bybit_tick if health.last_binance_tick > 0 and
binance_silence > max_feed_silence: logger.error(f"Binance feed silent for
{binance_silence:.0f}s") kill_switch.trigger(f"Binance feed silent
{binance_silence:.0f}s") break if health.last_bybit_tick > 0 and
bybit_silence > max_feed_silence: logger.error(f"Bybit feed silent for
{bybit_silence:.0f}s") kill_switch.trigger(f"Bybit feed silent
{bybit_silence:.0f}s") break # ตรวจสอบ order errors if health.order_errors >=
5: kill_switch.trigger( f"Too many order errors: {health.order_errors}" )
break logger.info( f"Health OK | equity={health.current_equity:,.0f} "
```



```
f"drawdown={health.drawdown_pct:.2%}" "  
f"order_errors={health.order_errors}" )
```

35.4 ระบบ Live Trading ครบวงจร

Running Example: BTC Pair Live Trading System

ประกอบทุก component จาก บท 31–34 เป็น production-ready system สมบูรณ์

```
import asyncio import logging import os import time import uuid from  
collections import deque from typing import Optional #  
===== #  
Configuration – โหลดจาก env var เสมอ #  
===== CONFIG =  
{ "symbol": "BTCUSDT", "interval": "1", # 1 minute candles "pair_window":  
60, # lookback candles "entry_zscore": 2.0, "exit_zscore": 0.5,  
"order_qty": 0.001, # BTC per trade "max_drawdown": -0.05, # หยุดถ้า drawdown  
เกิน 5% "max_feed_silence": 30.0, # หยุดถ้าไม่มี feed 30s "initial_capital":  
10_000.0, # USDT "testnet": True, # เปลี่ยนเป็น False เมื่อพร้อม live "log_level":  
"INFO", "log_file": "btc_pair_live.log", } class BTCPairLiveSystem: ""  
Live trading system สำหรับ BTC pair strategy Binance/Bybit spread → Z-score →  
market order "" def __init__(self, config: dict): self.config = config  
self.logger = setup_logging( config["log_level"], config.get("log_file") )  
self.kill_switch = KillSwitch() self.health = SystemHealth( peak_equity =  
config["initial_capital"], current_equity = config["initial_capital"], )  
self.order_tracker = OrderTracker() # Price buffers w =  
config["pair_window"] self._binance_closes: deque[float] = deque(maxlen=w)  
self._bybit_closes: deque[float] = deque(maxlen=w) # Position state  
self._position: Optional[str] = None # "LONG", "SHORT", หรือ None  
self._shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=1000) #  
—— Callbacks —— def  
_on_binance_candle(self, candle: Candle): self.health.last_binance_tick =  
time.time() def _on_bybit_candle(self, candle: Candle):  
self.health.last_bybit_tick = time.time() # — Signal Logic  
def _compute_zscore(self) ->  
Optional[float]: ""คำนวณ z-score ของ spread Binance-Bybit"" w =  
self.config["pair_window"] if (len(self._binance_closes) < w or  
len(self._bybit_closes) < w): return None import statistics spreads = [ b -  
y for b, y in zip( list(self._binance_closes), list(self._bybit_closes) ) ]  
mean = statistics.mean(spreads) stdev = statistics.stdev(spreads) if stdev  
== 0: return None return (spreads[-1] - mean) / stdev def _get_signal(self,  
zscore: float) -> str: entry = self.config["entry_zscore"] exit_ =  
self.config["exit_zscore"] if zscore > entry: return "SHORT" if zscore < -  
entry: return "LONG" if abs(zscore) < exit_: return "EXIT" return "HOLD" #  
—— Order Execution —— async def  
_execute_signal(self, signal: str, client: BybitOrderClient): ""แปลง signal  
เป็น order"" if signal == "HOLD": return if signal == self._position: return  
# อยู่ใน position นี้อยู่แล้ว qty = self.config["order_qty"] sym =
```

```

self.config["symbol"] coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}" if
signal in ("LONG", "SHORT"): # ปิด position เดิมก่อน (ถ้ามี) if self._position is
not None: close_side = (OrderSide.SELL if self._position == "LONG" else
OrderSide.BUY) close_coid = f"CLOSE-{uuid.uuid4().hex[:8].upper()}"
close_order = await client.place_market_order( sym, close_side, qty,
close_coid ) self.order_tracker.track(close_order) if close_order.status ==
OrderStatus.ERROR: self.health.order_errors += 1 # เปิด position ใหม่ side =
OrderSide.BUY if signal == "LONG" else OrderSide.SELL order = await
client.place_market_order(sym, side, qty, coid)
self.order_tracker.track(order) if order.status in (OrderStatus.OPEN,
OrderStatus.FILLED): self._position = signal self.logger.info( f"Position
opened: {signal}", extra={"symbol": sym, "order_id": coid, "side":
side.value} ) else: self.health.order_errors += 1
self.logger.error( f"Order failed: {order.reject_reason}",
extra={"order_id": coid} ) elif signal == "EXIT" and self._position is not
None: side = (OrderSide.SELL if self._position == "LONG" else
OrderSide.BUY) order = await client.place_market_order(sym, side, qty,
coid) self.order_tracker.track(order) if order.status in (OrderStatus.OPEN,
OrderStatus.FILLED): self._position = None self.logger.info( "Position
closed", extra={"symbol": sym, "order_id": coid} ) else:
self.health.order_errors += 1 # — Main Consumer Loop

————— async def _signal_loop(self, client:
BybitOrderClient): """ประมวลผล candle จาก queue และส่ง order""" while not
self.kill_switch.is_set: try: candle = await
asyncio.wait_for( self._shared_queue.get(), timeout=1.0 ) except
asyncio.TimeoutError: continue # เพิ่มเฉพาะ closed candle if candle.is_closed:
if candle.exchange == "binance": self._binance_closes.append(candle.close)
elif candle.exchange == "bybit": self._bybit_closes.append(candle.close)
zscore = self._compute_zscore() if zscore is not None: signal =
self._get_signal(zscore) self.logger.debug( f"zscore={zscore:.3f}
signal={signal}" ) if not self.kill_switch.is_set: await
self._execute_signal(signal, client) self._shared_queue.task_done() # —
Graceful Shutdown ————— async def
_shutdown(self, client: BybitOrderClient): """ปิด position ทั้งหมดก่อน
shutdown""" self.logger.warning("Graceful shutdown: ปิด open positions...")
if self._position is not None: side = (OrderSide.SELL if self._position ==
"LONG" else OrderSide.BUY) coid = f"SHUTDOWN-{uuid.uuid4().hex[:
8].upper()}" order = await
client.place_market_order( self.config["symbol"], side,
self.config["order_qty"], coid ) self.order_tracker.track(order)
self.logger.info( f"Shutdown order: {order.status.name}",
extra={"order_id": coid} ) self._position = None
self.logger.info( f"Shutdown complete | "
f"total_orders={len(self.order_tracker._orders)} | "
f"drawdown={self.health.drawdown_pct:.2%}" ) # — Run

————— async def run(self):
"""เฝ้าดูการทำงาน live""" loop = asyncio.get_event_loop()
self.kill_switch.setup_signal_handlers(loop) self.logger.info("Starting BTC
Pair Live System") self.logger.info(f"Config: {self.config}") # WebSocket
clients binance_client = BinanceWebSocketClient( symbol =
self.config["symbol"], interval = f"{self.config['interval']}m", on_candle
= self._on_binance_candle, queue = self._shared_queue, ) bybit_client =
BybitWebSocketClient( symbol = self.config["symbol"], interval =

```

```

self.config["interval"], on_candle = self._on_bybit_candle, queue =
self._shared_queue, ) async with
BybitOrderClient(testnet=self.config["testnet"]) as order_client: try:
await asyncio.gather( binance_client.run(), bybit_client.run(),
self._signal_loop(order_client), health_check_loop( self.health,
self.kill_switch, max_drawdown = self.config["max_drawdown"],
max_feed_silence = self.config["max_feed_silence"], ),
self.kill_switch.wait(), # รอจนกว่า kill switch return_exceptions=True, )
finally: binance_client.stop() bybit_client.stop() await
self._shutdown(order_client) # — Entry point
_____ if __name__ == "__main__":
system = BTCPairLiveSystem(CONFIG) asyncio.run(system.run())

```

35.5 Pre-Flight Checklist ก่อน Go Live

```

import asyncio import os import aiohttp async def preflight_check(config:
dict) -> bool: """ ตรวจสอบความพร้อมก่อนเริ่ม live trading คืน True เฉพาะเมื่อผ่านทุก check
""" results = {} logger = logging.getLogger("live_trading.preflight") # 1.
ตรวจ API Keys api_key = os.environ.get("BYBIT_API_KEY", "") api_secret =
os.environ.get("BYBIT_API_SECRET", "") results["api_keys"] = bool(api_key
and api_secret and len(api_key) > 10 and len(api_secret) > 10) if not
results["api_keys"]: logger.error("FAIL: BYBIT_API_KEY หรือ BYBIT_API_SECRET
ไม่ได้ตั้งค่า") # 2. ตรวจการเชื่อมต่อ exchange try: async with aiohttp.ClientSession()
as session: url = ("https://api-testnet.bybit.com/v5/market/time" if
config.get("testnet") else "https://api.bybit.com/v5/market/time") async
with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as r: data =
await r.json() results["connectivity"] = data.get("retCode") == 0 except
Exception as e: results["connectivity"] = False logger.error(f"FAIL: ไม่
สามารถเชื่อมต่อ exchange: {e}") # 3. ตรวจ balance if results.get("api_keys") and
results.get("connectivity"): try: async with
BybitOrderClient(testnet=config.get("testnet", True)) as client: resp =
await client._request("GET", "/v5/account/wallet-balance", {"accountType":
"UNIFIED"}) if resp.get("retCode") == 0: coins = resp["result"]["list"][0]
["coin"] usdt = next( (float(c["availableToWithdraw"]) for c in coins if
c["coin"] == "USDT"), 0.0 ) min_balance = config.get("initial_capital",
1000) * 0.5 results["balance"] = usdt >= min_balance if not
results["balance"]: logger.error( f"FAIL: USDT balance {usdt:.2f} <
required {min_balance:.2f}" ) else: logger.info(f"OK: USDT balance =
{usdt:.2f}") else: results["balance"] = False except Exception as e:
results["balance"] = False logger.error(f"FAIL: ดึง balance ไม่ได้: {e}") # 4.
ตรวจ config cfg_ok = ( config.get("order_qty", 0) > 0 and
config.get("max_drawdown", 0) < 0 and config.get("entry_zscore", 0) >
config.get("exit_zscore", 0) > 0 ) results["config"] = cfg_ok if not
cfg_ok: logger.error("FAIL: Config ไม่ถูกต้อง") # 5. สรุป all_pass =
all(results.values()) print("\n=== Pre-Flight Check ===") for name, ok in
results.items(): status = "✓ PASS" if ok else "✗ FAIL" print(f" {name:20s}:
{status}") print(f"\n{'→ READY TO LAUNCH' if all_pass else '→ FIX ISSUES
FIRST'}\n") return all_pass # asyncio.run(preflight_check(CONFIG))

```

Deployment Checklist — ก่อน Go Live จริง

- ทดสอบบน **Testnet** ≥ 1 สัปดาห์ — ให้ระบบวิ่งตลอดเวลา ดู drawdown และ error logs
- **Paper trading** บน **real feed** — รับ feed จริงแต่ไม่ส่ง order จริง เพื่อตรวจ latency
- ตั้ง **max position size** เล็กๆ ตอนเริ่มต้น เช่น 0.001 BTC ก่อน scale ขึ้น
- มี **monitoring** ภายนอก — อย่าเชื่อแค่ log ของตัวเอง ใช้ Telegram bot หรือ Grafana alert
- **Deploy** บน **VPS** ใกล้ **exchange** — Singapore หรือ Tokyo เพื่อ latency ต่ำ
- ตั้ง **systemd** หรือ **Docker** ให้ restart อัตโนมัติถ้า process crash
- **Log rotation** — log file โตได้มาก ตั้ง logrotate หรือ size limit

► แบบฝึกหัด: เพิ่ม **Telegram alert** เมื่อ **kill switch** ทำงาน หรือเมื่อ **fill order** ใหญ่

```
import aiohttp import os TELEGRAM_BOT_TOKEN =
os.environ.get("TELEGRAM_BOT_TOKEN", "") TELEGRAM_CHAT_ID =
os.environ.get("TELEGRAM_CHAT_ID", "") async def
send_telegram(message: str) -> bool: """ส่ง alert ผ่าน Telegram Bot""" if
not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID: return False url =
f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage" try:
async with aiohttp.ClientSession() as session: async with
session.post(url, json={ "chat_id": TELEGRAM_CHAT_ID, "text": message,
"parse_mode": "HTML" }, timeout=aiohttp.ClientTimeout(total=5)) as
resp: return resp.status == 200 except Exception: return False # ใน
KillSwitch.trigger(): async def trigger_with_alert(self, reason: str =
>manual"): self._reason = reason self._event.set()
logger.critical(f"KILL SWITCH TRIGGERED: {reason}") await
send_telegram( f"    KILL SWITCH\n" f"Reason: {reason}\n" f"Time:
{datetime.utcnow().isoformat()}Z" ) # ใน _execute_signal() หลัง order
filled: async def notify_fill(order: Order): msg = (f"    Order
Filled\n" f"Symbol: {order.symbol}\n" f"Side: {order.side.value}\n"
f"Qty: {order.filled_qty}\n" f"AvgPrice: {order.avg_price:,.2f}")
await send_telegram(msg)
```

จบหนังสือ — เส้นทางต่อไป

การเดินทางจาก **PART 0** ถึง **PART VI**

คุณผ่านการเรียนรู้มาไกลมาก — ตั้งแต่ Python พื้นฐานจนถึงระบบ live trading อัตโนมัติ

สรุปทุกอย่างที่ได้เรียนในหนังสือเล่มนี้:

Part	เนื้อหา	สิ่งที่ได้
Part 0	Python Foundations	Syntax, data types, functions, comprehensions
Part I	Data Wrangling	pandas, numpy, matplotlib — ดึง/ทำความสะอาด/visualize ข้อมูลตลาด
Part II	Math Tools	Rolling stats, z-score, cointegration, pair trading signal
Part III	OOP	Classes, ABC, Strategy/Observer/Factory patterns — system ที่ขยายได้
Part IV	Backtesting	Walk-forward, slippage, commission, overfitting, Sharpe/drawdown
Part V	AI-Assisted Coding	Prompt engineering, code generation, audit checklist, quant libraries, full strategy with AI
Part VI	Async & Live Trading	asyncio, WebSocket, order execution, kill switch, monitoring

เส้นทางการพัฒนาในฐานะ Quant Developer:

[หนังสือ
นี้] Part 0-I → Part II-III → Part IV-V → Part VI Python basics Math + OOP Backtest Live System | | | ▼ ▼ ▼ ▼ [ขั้นต่อไป] NumPy/Pandas statsmodels vectorbt/ production เชี่ยวชาญ PyMC (Bayes) zipline deployment Nautilus cloud infra

แนะนำขั้นตอนต่อไปหลังจบหนังสือ

1. Backtesting เชิงลึก

- **vectorbt** — vectorized backtesting เร็วกว่า pandas loop 100x
- **Nautilus Trader** — professional backtester + live trading framework ที่ใช้ใน production
- **Zipline-reloaded** — backtester ของ Quantopian ที่ยังมี community ดูแล

2. Statistical Methods เพิ่มเติม

- **Bayesian Inference** ด้วย PyMC — ประเมิน parameter uncertainty แทน point estimate
- **Kalman Filter** — dynamic hedge ratio ที่ปรับตามเวลา (statsmodels มี DLM)
- **Regime Detection** ด้วย Hidden Markov Model — ตรวจสอบว่าตลาดอยู่ใน trend/mean-revert/volatile

3. Machine Learning for Finance

- **Feature Engineering** — microstructure features, order book imbalance, funding rate
- **Gradient Boosting** (XGBoost/LightGBM) สำหรับ signal classification
- **LSTM / Transformer** สำหรับ time series — แต่ระวัง overfitting
- **Reinforcement Learning** (Stable-Baselines3) — ให้ model เรียนรู้ position sizing

4. Infrastructure & Operations

- **Docker + docker-compose** — containerize trading system
- **TimescaleDB** — PostgreSQL extension สำหรับ time series data ใน production
- **Grafana + Prometheus** — monitoring dashboard สำหรับ live system
- **Redis Streams** — message queue สำหรับ high-throughput tick data

5. หนังสือและแหล่งเรียนรู้แนะนำ

- Advances in Financial Machine Learning — Marcos Lopez de Prado (อ่านหลังจากมีพื้นฐาน ML)
- Algorithmic Trading — Ernest Chan (practical, Python examples)
- Active Portfolio Management — Grinold & Kahn (classic quant text)
- QuantLib Python — pricing library สำหรับ options และ derivatives
- Quantopian Lectures (archive) — beginner-friendly quant tutorials

คำเตือนสุดท้าย — จากนักพัฒนาถึงนักพัฒนา

หนังสือนี้สอนวิธี สร้าง trading system — ไม่ใช่วิธีรวยจากตลาด

- Strategy ที่ backtest ได้ดีส่วนมากไม่ได้ผลใน live — นั่นคือความจริงของ quant
- Market เปลี่ยนไปตลอด — strategy ที่ดีวันนี้อาจไม่ดีพรุ่งนี้
- การ risk management และ position sizing สำคัญกว่า entry signal มาก
- ไม่มี **strategy** ไหนที่ไม่มีความเสี่ยง — จัดการความเสี่ยงด้วยระบบ ไม่ใช่ด้วยความหวัง

จงสร้างระบบที่ fail safely, log everything, และ never risk what you can't afford to lose